# Business Requirements Document (BRD)

**Project Name:** EstateFlow (Comprehensive Property Management System)
**Document Version:** 2.0
**Date:** July 2026

---

## 1. Executive Summary

EstateFlow is a state-of-the-art, web-based Property Management System (PMS) engineered to overhaul the traditional, manual processes of real estate rental management. By providing a centralized, secure, and user-friendly digital ecosystem, EstateFlow connects property owners, assigned property managers (admins), and tenants on a single platform.

The traditional rental market suffers from fragmented communication, opaque rent tracking, cumbersome physical KYC verification, paper-based lease agreements, and inefficient maintenance resolution. EstateFlow directly addresses these pain points by offering an integrated suite of tools including role-based dashboards, automated digital lease generation with e-signatures, seamless payment gateway integration (Razorpay), automated cron-job-based rent reminders, a robust maintenance ticketing system, and interactive notice boards.

This Business Requirements Document (BRD) serves as the foundational blueprint for the EstateFlow project. It details the comprehensive functional and non-functional requirements, target audience profiles, system architecture, detailed user stories, data schemas, and API contracts necessary for the successful development and deployment of the platform.

---

## 2. Project Overview & Scope

### 2.1 Background and Problem Statement

In the current real estate landscape, property owners managing multiple units face significant administrative overhead. Rent collection is often manual and prone to delays. Tenant onboarding requires physical document submission and manual verification, leading to slow turnaround times. Lease agreements are physical documents that are easily lost or damaged. When maintenance issues arise, tenants rely on phone calls or WhatsApp messages, which are difficult to track and prioritize, leading to tenant dissatisfaction.

### 2.2 Objective

The primary objective of EstateFlow is to build a robust, scalable web application using the MERN stack (MongoDB, Express.js, React.js, Node.js) that automates the end-to-end lifecycle of property management.

### 2.3 Scope of the MVP (Minimum Viable Product)

**In-Scope:**

- Secure User Authentication and Authorization (JWT-based).
- Multi-tier Role-Based Access Control (RBAC) distinguishing between Owner, Admin, and User (Tenant) roles.
- Property and Unit lifecycle management (CRUD operations).
- Tenant Onboarding & KYC submission (Document upload and validation).
- Automated Lease Generation (PDF generation with digital signatures).
- Payment Gateway Integration (Razorpay for security deposit and rent).
- Monthly Rent Management (Automated generation of rent dues).
- Automated Email Reminders (Using node-cron for upcoming and overdue rent).
- Maintenance Request Ticketing System (Categorization, priority, and status tracking).
- Notice Board and Announcement Broadcasting (Owner to Tenants).
- Property Reviews and Ratings (Tenant feedback system).
- Responsive UI/UX tailored for desktop and mobile web browsers.

**Out-of-Scope (Deferred for Future Phases):**

- Native Mobile Applications (iOS/Android).
- Complex Third-Party Accounting Software Integration (e.g., QuickBooks, Xero).
- Smart Home IoT Integration (e.g., automated door locks, smart thermostats).
- AI-driven dynamic pricing algorithms.

---

## 3. Target Audience and Stakeholder Profiles

### 3.1 Property Owners

- **Profile**: Individuals or corporate entities owning multiple residential or commercial rental properties.
- **Pain Points**: Lack of visibility into overall portfolio performance, difficulty tracking which tenants are late on rent, inability to effectively oversee property managers.
- **Goals**: Maximize occupancy rates, ensure timely rent collection, gain high-level financial and operational insights via analytics dashboards, and delegate day-to-day operations to Admins securely.

### 3.2 Property Managers (Admins)

- **Profile**: Staff or contractors hired by Property Owners to oversee specific buildings or units.
- **Pain Points**: Overwhelmed by tenant maintenance calls, manual tracking of lease renewals, and physical rent collection.
- **Goals**: efficiently handle day-to-day operations, approve KYC documents swiftly, resolve maintenance tickets methodically, and ensure tenant satisfaction.

### 3.3 Tenants (Users)

- **Profile**: Individuals (students, professionals, families) renting rooms, apartments, or PG accommodations.
- **Pain Points**: Hidden fees, lack of transparency in lease terms, difficulty paying rent securely online, and slow response times to maintenance issues.
- **Goals**: Easily browse verified properties, submit KYC digitally, sign leases without visiting an office, pay rent via preferred digital methods securely, raise maintenance requests easily, and stay informed via official notices.

---

## 4. Comprehensive Functional Requirements

### 4.1 Authentication and Role Management (RBAC)

- **FR 1.1 - User Registration**: The system shall allow users to register using their full name, email address, password, and phone number.
- **FR 1.2 - Password Security**: Passwords must be hashed using bcrypt with a minimum salt round of 10 before being stored in the database.
- **FR 1.3 - Login Authentication**: The system shall authenticate users via email and password, returning a JWT valid for a configurable duration (e.g., 30 days).
- **FR 1.4 - Role Assignment**: By default, new registrants are assigned the 'User' role. The 'Owner' role must be provisioned manually by super-admins (database level for MVP).
- **FR 1.5 - Admin Creation**: Owners shall have the ability to create 'Admin' accounts from their dashboard and assign them to specific properties.
- **FR 1.6 - Route Protection**: The frontend and backend shall enforce route protection, ensuring users cannot access admin/owner dashboards or APIs without the appropriate JWT role claims.

### 4.2 Owner Dashboard & Analytics

- **FR 2.1 - Financial Overview**: Owners shall view a consolidated dashboard displaying Total Monthly Revenue, Total Expected Revenue, and Total Advance Collected.
- **FR 2.2 - Occupancy Metrics**: The dashboard shall display a visual chart (e.g., Pie Chart) showing the ratio of Occupied vs. Vacant units across the portfolio.
- **FR 2.3 - Lease Tracking**: The system shall list all expiring leases within the next 30, 60, and 90 days.
- **FR 2.4 - Refund Management**: Owners shall view a list of tenants who have requested to leave, along with their bank details, to process security deposit refunds manually.
- **FR 2.5 - Admin Management**: Owners shall view, edit, and revoke access for property managers (Admins).

### 4.3 Property & Unit Management

- **FR 3.1 - Property Creation**: Owners shall be able to create new properties by providing: Name, Type (Apartment, DailyRoom, PG), Address, City, State, Base Rent, Advance/Deposit Amount, Amenities (comma-separated), Description, and uploading Images.
- **FR 3.2 - Automated Unit Generation**: Upon creating a property, the owner specifies the number of rooms. The system shall automatically generate unit records (e.g., 101, 102, 103) with status set to 'Available'.
- **FR 3.3 - Property Editing**: Owners can edit property details. If the number of rooms is increased, new units are generated. If decreased, unoccupied units are deleted (system must prevent deletion of occupied units).
- **FR 3.4 - Property Status**: A property's overall status shall dynamically reflect 'Available' if at least one unit is vacant, or 'Full' if all units are occupied.

### 4.4 Tenant Browsing and Booking Flow

- **FR 4.1 - Public Listings**: Users (even unauthenticated) can browse the `PropertiesList` page, search by location/name, and sort by price.
- **FR 4.2 - Detailed View**: Users can view a specific property's details, including image galleries, amenities, base rent, and a grid of available rooms.
- **FR 4.3 - Booking Initiation**: Clicking "Book Now" on an available room shall prompt unauthenticated users to log in/register.
- **FR 4.4 - KYC Submission (Step 1)**: The user must fill out a KYC form requiring: Current Address, Phone Number, Passport Photo, Aadhaar Card (Image/PDF), and Company/Student ID.
- **FR 4.5 - DailyRoom Logic**: If the property is a 'DailyRoom', the user must select Check-in and Check-out dates. The system calculates the total payable amount based on the number of days.
- **FR 4.6 - Standard Lease Logic**: For standard leases, the system calculates a prorated rent amount based on the remaining days in the current month, plus the security deposit.
- **FR 4.7 - Digital Signature (Step 2)**: The user must draw their signature on an HTML5 Canvas to agree to the lease terms.
- **FR 4.8 - Payment (Step 3)**: The user proceeds to a Razorpay checkout modal to pay the calculated total amount.

### 4.5 Payment Processing & Razorpay Integration

- **FR 5.1 - Order Creation**: The backend shall create a Razorpay Order (`rp_order_id`) for the exact amount before opening the frontend modal.
- **FR 5.2 - Payment Verification**: Upon successful payment, the frontend sends the payment signature to the backend. The backend shall verify the signature using the Razorpay Secret using HMAC SHA256.
- **FR 5.3 - Transaction Recording**: Once verified, the backend shall create a Rent/Payment document marking the initial payment as 'Paid'.
- **FR 5.4 - Lease Activation**: The backend shall create a Lease document, set the Unit status to 'Occupied', and update the User's KYC URLs.

### 4.6 Lease Document Generation

- **FR 6.1 - PDF Generation**: The system shall utilize `jsPDF` (or a backend equivalent like `pdfkit`) to generate a formal Lease Agreement.
- **FR 6.2 - Document Contents**: The PDF must include the Owner's name, Tenant's name, Property address, Unit number, Rent amount, Deposit amount, Start Date, standard terms and conditions, and embed the Tenant's digital signature image.
- **FR 6.3 - Accessibility**: The generated lease details are stored in the database, and the tenant can download the PDF receipt/agreement at any time from their dashboard.

### 4.7 Tenant Dashboard

- **FR 7.1 - Active Lease Overview**: Tenants shall see their current property name, unit number, monthly rent, and initial deposit paid.
- **FR 7.2 - Next Rent Due**: A dedicated card shall show the upcoming rent amount, due date (1st of the month), and a "Pay Rent Now" button integrated with Razorpay.
- **FR 7.3 - Payment History**: A chronological list of past rent payments with the ability to download PDF receipts for each transaction.
- **FR 7.4 - Leave Room Request**: Tenants can click "Leave Room & Claim Refund", which opens a modal requiring Bank Name, Account Number, and IFSC Code. Submitting this changes the lease status to 'Terminated' and refund status to 'Pending'.

### 4.8 Maintenance Ticketing System

- **FR 8.1 - Ticket Creation**: Tenants can raise maintenance tickets categorized into: Plumbing, Electrical, Appliance, Carpentry, Other.
- **FR 8.2 - Description & Priority**: Tenants must provide a text description. The system assigns a default 'Medium' priority (expandable in future).
- **FR 8.3 - Ticket Tracking**: Tenants can view a list of their past and current tickets along with their status (Pending, In Progress, Resolved, Closed).
- **FR 8.4 - Admin Resolution**: Admins/Owners view a centralized list of all tickets for their properties and can update the status via a dropdown menu.

### 4.9 Notice Board & Broadcasting

- **FR 9.1 - Notice Creation**: Owners/Admins can select a property, enter a Title (e.g., "Water Supply Cut"), and Content to create a Notice.
- **FR 9.2 - Notice Display**: Notices are displayed prominently on the Tenant Dashboard for users residing in the selected property.
- **FR 9.3 - Automated Emails (Notification)**: Creating a notice shall trigger a backend function that queries all active tenants in that property and sends them an

automated email using `nodemailer` (e.g., SendGrid/SMTP).

### 4.10 Reviews and Ratings

- **FR 10.1 - Review Submission**: Tenants with an active (or past) lease can use a form on their dashboard to select a 1-5 star rating and write a comment about the property.
- **FR 10.2 - Public Display**: The `PropertyDetails` page shall fetch all reviews for that property and display them at the bottom of the page, showing the user's initial, comment, and star rating.
- **FR 10.3 - Aggregation**: The system shall calculate the Average Rating and Total Review Count and display this data on the `PropertiesList` cards.

### 4.11 Automated Rent Reminders (Cron Jobs)

- **FR 11.1 - Monthly Rent Generation**: On the 28th of every month, a cron job shall generate new 'Pending' rent records for the upcoming month for all active leases.
- **FR 11.2 - Upcoming Reminder Job**: Scheduled to run on the 1st of every month at 08:00 AM. It queries all 'Pending' rent records and sends a polite "Rent Due Today" email with a payment link to the respective tenants.
- **FR 11.3 - Overdue Reminder Job**: Scheduled to run on the 5th of every month at 08:00 AM. It queries all rent records that are still 'Pending' past the 4th of the month, updates their status to 'Overdue', and sends an urgent "Overdue Rent Warning" email to the tenants. Owners/Admins are also notified.

---

## 5. Non-Functional Requirements (NFRs)

### 5.1 Performance & Scalability

- **Page Load Time**: The frontend application should load in under 2 seconds on standard broadband connections.
- **API Response Time**: Backend API responses should average less than 300ms under normal load.
- **Scalability**: The Node.js server shall be stateless, allowing for horizontal scaling using load balancers (e.g., NGINX, AWS ALB) if traffic increases.
- **Database Indexing**: MongoDB collections must have indexes on frequently queried fields (e.g., `user_id`, `property_id`, `email`) to ensure fast read operations.

### 5.2 Security & Compliance

- **Data Encryption**: All data transmitted between client and server must use TLS/SSL (HTTPS).
- **Authentication**: JWTs must be signed with a strong, rotating secret key and should not contain highly sensitive PII in the payload.
- **Payment Compliance**: EstateFlow does not store credit card or UPI details. All payment processing relies entirely on Razorpay's PCI-DSS compliant infrastructure.
- **Input Validation**: All API inputs must be validated on the backend (using libraries like Joi or express-validator) to prevent NoSQL injection and XSS attacks.
- **File Uploads**: KYC document uploads must be restricted to image formats and PDFs, with size limits (e.g., max 5MB per file) to prevent server storage abuse.

### 5.3 Reliability, Availability, and Maintainability

- **Uptime Target**: The system is designed for 99.9% uptime.
- **Error Handling**: The backend must implement a global error handling middleware to catch unhandled promise rejections and format error responses consistently.
- **Logging**: Implement robust logging (e.g., Winston or Morgan) for all API requests, payment webhooks, and cron job executions for auditing and debugging.
- **Backups**: MongoDB databases should be configured for automated daily backups.

### 5.4 UI/UX & Accessibility

- **Responsive Design**: The React frontend (using Tailwind CSS) must adapt fluidly to mobile (320px width), tablet (768px), and desktop (1024px+) screens.
- **Accessibility (a11y)**: Semantic HTML tags must be used. Images must have `alt` tags. Contrast ratios must meet WCAG AA standards.
- **Feedback Mechanisms**: The UI must provide clear feedback via Toast notifications for successes (e.g., "Review Submitted") and errors (e.g., "Payment Failed"). Loading spinners must be used during async operations.

---

## 6. Detailed System Architecture

### 6.1 Technology Stack

- **Frontend Framework**: React.js (Bootstrapped with Vite for faster HMR and builds).
- **State Management**: React Context API (for Auth state) and React Hooks (useState, useEffect for local component state).
- **Routing**: React Router DOM v6.
- **Styling**: Tailwind CSS (Utility-first CSS framework).
- **Icons & Charts**: `lucide-react` for iconography, `recharts` for Owner Dashboard analytics.
- **Backend Framework**: Node.js with Express.js.
- **Database**: MongoDB Atlas (Cloud-hosted NoSQL DB).
- **ORM / ODM**: Mongoose.
- **Authentication**: `jsonwebtoken` (JWT) and `bcryptjs`.
- **File Uploads**: `multer` (Handling `multipart/form-data`) integrated with Cloudinary or local file system.
- **Payment Gateway**: Razorpay Node SDK.
- **Background Jobs**: `node-cron`.
- **Email Service**: `nodemailer` (SMTP transport).

### 6.2 Architecture Diagram Overview

1. **Client Browser** (React SPA) sends RESTful HTTP requests to the Backend API.
2. **Backend API Gateway** (Express.js) routes requests.
3. **Auth Middleware** intercepts protected routes, verifies the JWT in the `Authorization: Bearer <token>` header.
4. **Controllers** execute business logic (e.g., calculating rent, verifying Razorpay signatures).
5. **Mongoose Models** interact with the MongoDB database to perform CRUD operations.
6. **Cron Server** (runs within the Node process or as a separate worker) checks time schedules and triggers controller functions to send emails via Nodemailer.

---

## 7. Data Models / Database Schema Dictionary

### 7.1 User Model (`User.js`)

| Field | Type | Modifiers | Description |
|---|---|---|---|

| | | | |
|---|---|---|---|
| `_id` | ObjectId | Auto | Unique identifier |
| `name` | String | Required | User's full name |
| `email` | String | Required, Unique | Email address used for login |
| `password` | String | Required | Bcrypt hashed password |
| `role` | String | Enum: ['User', 'Admin', 'Owner'], Default: 'User' | Access control level |
| `phone` | String | Optional | Contact number (collected during KYC) |
| `address` | String | Optional | Permanent address (collected during KYC) |
| `kyc_details` | Array of Strings | Optional | URLs to uploaded Aadhaar, ID, Photo |

### 7.2 Property Model (`Property.js`)

| Field | Type | Modifiers | Description |
|---|---|---|---|
| `owner_id` | ObjectId | Ref: 'User', Required | The owner of the property |
| `assigned_admin_id` | ObjectId | Ref: 'User' | The manager assigned to this property |
| `name` | String | Required | E.g., "Sunset Apartments" |
| `type` | String | Enum: ['Apartment', 'DailyRoom', 'PG', 'Commercial'] | Category of property |
| `address`, `city`, `state` | String | Required | Location details |
| `rent_amount` | Number | Required | Base monthly rent or daily rate |
| `deposit_amount` | Number | Required | Security advance required |
| `amenities` | Array of Strings | | E.g., ["WiFi", "AC", "Gym"] |
| `description` | String | | Detailed text about the property |
| `images` | Array of Strings | | URLs to property photos |
| `status` | String | Enum: ['Available', 'Full'] | Computed or set based on unit occupancy |

### 7.3 Unit Model (`Unit.js`)

| Field | Type | Modifiers | Description |
|---|---|---|---|
| `property_id` | ObjectId | Ref: 'Property', Required | Parent property |
| `unit_no` | String | Required | Room or apartment number (e.g., "101") |
| `rent_amount` | Number | Required | Specific rent for this unit (defaults to property base rent) |
| `status` | String | Enum: ['Available', 'Occupied', 'Maintenance'], Default: 'Available' | Current state of the room |

### 7.4 Lease Model (`Lease.js`)

| Field | Type | Modifiers | Description |
|---|---|---|---|
| `property_id` | ObjectId | Ref: 'Property' | Associated property |
| `unit_id` | ObjectId | Ref: 'Unit' | Associated room |
| `user_id` | ObjectId | Ref: 'User' | The tenant |
| `start_date` | Date | Required | Move-in date |
| `end_date` | Date | Optional | Move-out date (required for DailyRoom) |
| `rent_amount` | Number | Required | Agreed upon rent |
| `deposit` | Number | Required | Agreed upon deposit |
| `status` | String | Enum: ['Active', 'Terminated', 'Expired'], Default: 'Active' | Lease status |
| `refund_status` | String | Enum: ['N/A', 'Pending', 'Processed'], Default: 'N/A' | Deposit refund status |
| `bank_details` | Object | | Contains `bank_name`, `account_number`, `ifsc_code` for refunds |

### 7.5 Rent (Payment) Model (`Rent.js`)

| Field | Type | Modifiers | Description |
|---|---|---|---|
| `lease_id` | ObjectId | Ref: 'Lease' | The associated lease |
| `user_id` | ObjectId | Ref: 'User' | The tenant paying |
| `property_id` | ObjectId | Ref: 'Property' | For easier aggregation by owner |
| `month` | String | Required | E.g., "July 2026" |
| `due_amount` | Number | Required | Amount required to be paid |
| `paid_amount` | Number | Default: 0 | Amount actually paid |
| `due_date` | Date | Required | When the rent is due |
| `status` | String | Enum: ['Pending', 'Paid', 'Overdue'], Default: 'Pending' | Payment status |
| `razorpay_payment_id` | String | | Transaction ID from Razorpay |
| `razorpay_order_id` | String | | Order ID from Razorpay |

### 7.6 Maintenance Model (`Maintenance.js`)

| Field | Type | Modifiers | Description |
|---|---|---|---|
| `property_id` | ObjectId | Ref: 'Property' | Where the issue occurred |
| `user_id` | ObjectId | Ref: 'User' | Tenant who reported the issue |
| `category` | String | Enum: ['Plumbing', 'Electrical', 'Appliance', 'Carpentry', 'Other'] | Issue type |
| `description` | String | Required | Details of the problem |
| `priority` | String | Enum: ['Low', 'Medium', 'High'], Default: 'Medium' | Urgency |
| `status` | String | Enum: ['Pending', 'In Progress', 'Resolved', 'Closed'], Default: 'Pending' | Ticket status |

### 7.7 Notice Model (`Notice.js`)

| Field | Type | Modifiers | Description |
|---|---|---|---|
| `property_id` | ObjectId | Ref: 'Property', Required | Target property for the broadcast |
| `title` | String | Required | Headline of the notice |
| `content` | String | Required | Body of the notice |
| `createdAt` | Date | Default: Date.now | Timestamp |

### 7.8 Review Model (`Review.js`)

| Field | Type | Modifiers | Description |
|---|---|---|---|
| property_id | ObjectId | Ref: 'Property', Required | Property being reviewed |
| user_id | ObjectId | Ref: 'User', Required | Tenant leaving the review |
| rating | Number | Required, Min: 1, Max: 5 | Star rating |
| comment | String | Required | Written feedback |
| createdAt | Date | Default: Date.now | Timestamp |

---

## 8. Detailed API Endpoints Dictionary

*Note: All private routes require `Authorization: Bearer <token>`.*

### Auth & Users (`/api/auth`)

- `POST /register`: Register a new User.
- `POST /login`: Authenticate and return JWT.
- `GET /me`: Get current logged-in user details.
- `PUT /profile`: Update user profile (used for saving KYC data post-upload).
- `POST /create-admin`: (Owner Only) Create a new admin account.
- `GET /admins`: (Owner Only) List all admins assigned by this owner.

### Properties (`/api/properties`)

- `GET /`: List all properties. If Owner/Admin, returns properties with stats (Occupancy, Revenue).
- `POST /`: (Owner Only) Create a new property and generate units.
- `GET /:id`: Get specific property details along with its units.
- `PUT /:id`: (Owner Only) Update property details.
- `DELETE /:id`: (Owner Only) Delete property and its units.

### Leases (`/api/leases`)

- `POST /book`: Finalize booking, create lease, mark unit as occupied.
- `GET /my-lease`: (User Only) Get the active lease for the logged-in tenant.
- `POST /:id/terminate`: (User Only) Initiate lease termination, providing bank details for refund.
- `GET /expiring`: (Owner/Admin) Get leases expiring within 30-90 days.

### Payments (`/api/payments`)

- `POST /create-order`: Generate a Razorpay order ID for a specific amount.
- `POST /verify`: Verify Razorpay signature after frontend payment succeeds.

### Rent (`/api/rent`)

- `GET /my-rent`: (User Only) Get the next pending rent due for the tenant.
- `GET /lease/:leaseId`: Get rent payment history for a specific lease.

### Maintenance (`/api/maintenance`)

- `POST /`: (User Only) Create a new maintenance ticket.
- `GET /my-requests`: (User Only) Get tickets created by the tenant.
- `GET /`: (Owner/Admin) Get all tickets for properties managed by the user.
- `PUT /:id`: (Owner/Admin) Update ticket status.

### Notices (`/api/notices`)

- `POST /`: (Owner/Admin) Broadcast a new notice and trigger automated emails.
- `GET /my-notices`: (User Only) Fetch notices for the property the user currently resides in.
- `GET /:propertyId`: (Owner/Admin) Fetch notices sent for a specific property.

### Reviews (`/api/reviews`)

- `POST /`: (User Only) Submit a rating and comment for a property.
- `GET /:propertyId`: Fetch all reviews and average rating for a specific property.

### Uploads (`/api/upload`)

- `POST /`: Handle `multipart/form-data` uploads via Multer and return file URLs.

---

## 9. Comprehensive User Stories & Acceptance Criteria

### Epic 1: Property Booking and Tenant Onboarding

**User Story 1.1**: As a prospective tenant, I want to browse available properties without logging in so that I can explore my options.

- *Acceptance Criteria*: `PropertiesList` page is publicly accessible. Search and sort filters work without a JWT token.

**User Story 1.2**: As a tenant, I want to upload my KYC documents securely during booking so that I comply with owner requirements.

- *Acceptance Criteria*: The booking modal enforces upload of Photo, Aadhaar, and ID. Phone number must be exactly 10 digits. The backend saves these to the user's profile upon successful payment.

**User Story 1.3**: As a tenant, I want to sign my lease digitally so that I don't have to print and scan documents.

- *Acceptance Criteria*: An HTML5 canvas allows drawing a signature. The signature is exported as a Base64 image and embedded into the final PDF generated by `jsPDF`.

### Epic 2: Payments and Rent Management

**User Story 2.1**: As a tenant, I want to pay my initial deposit and rent securely via a payment gateway.

- *Acceptance Criteria*: Razorpay modal opens with the correct prorated amount. Successful payment creates a `Rent` record marked as 'Paid'.

**User Story 2.2**: As a tenant, I want to receive email reminders when my rent is due so that I don't miss the deadline.

- *Acceptance Criteria*: Cron job runs on the 1st of the month. Tenant receives an email with the subject "Rent Due Today".

**User Story 2.3**: As an owner, I want to see a list of tenants who are overdue on rent so I can take action.

- *Acceptance Criteria*: Cron job on the 5th marks pending rent as 'Overdue'. Owner dashboard reflects these in the financial metrics.

### Epic 3: Communication and Feedback

**User Story 3.1**: As an owner, I want to broadcast notices (e.g., maintenance schedules) to all tenants in a building simultaneously.

- *Acceptance Criteria*: Owner uses the Notice form. Tenants see the notice on their dashboard. Tenants receive an immediate email alert.

**User Story 3.2**: As a tenant, I want to leave a review for my rental experience so that future tenants are informed.

- *Acceptance Criteria*: Tenant fills out the Review form. Property Details page publicly shows the average star rating and individual comments.

---

## 10. Implementation Milestones and Timelines

*Assuming an Agile 4-Sprint cycle (2 weeks per Sprint).*

**Sprint 1: Foundation and Property Management**

- Setup MERN repository, CI/CD, and MongoDB Atlas.
- Implement User Authentication, RBAC, and Profile management.
- Build Owner/Admin Property & Unit CRUD APIs.
- Develop frontend `OwnerDashboard` (Properties tab) and public `PropertiesList`.

**Sprint 2: Booking Engine and Payments**

- Integrate AWS S3 / local Multer for file uploads (KYC).
- Develop multi-step booking modal (KYC -> Signature -> Payment).
- Integrate Razorpay API for Order creation and verification.
- Implement `jsPDF` lease generation.

**Sprint 3: Tenant Dashboard and Core Operations**

- Develop `UserDashboard` showing active lease and payment history.
- Implement Rent tracking models and APIs.
- Build Maintenance Ticketing system (Frontend and Backend).
- Build the "Leave Room" / Refund workflow.

**Sprint 4: Automation, Communication, and Polish**

- Implement Notice Board feature and email dispatch via Nodemailer.
- Implement Reviews & Ratings system.
- Setup `node-cron` jobs for monthly rent generation and email reminders.
- Final QA, Security Audits, and Production Deployment.

---

## 11. Future Enhancements (Post-MVP Phase 2)

1. **In-App Real-Time Chat**: Replace reliance on external communication by implementing WebSockets (Socket.io) for direct messaging between tenants and admins.
2. **Expense and ROI Tracking**: A dedicated accounting module for owners to log property expenses (taxes, repairs, utilities) against revenue to calculate precise Return on Investment (ROI).
3. **Mobile Application (React Native)**: Native iOS and Android apps for tenants to receive push notifications for rent and maintenance updates, replacing email reliance.
4. **Smart Lock IoT Integration**: Integration with digital smart locks to automatically grant access to the building when a lease begins, and revoke access when a lease is terminated or rent is severely overdue.
5. **AI-Driven Dynamic Pricing**: Machine learning algorithms that analyze local market trends, seasonal demand, and property vacancy duration to suggest optimal rent prices to owners.
6. **Multi-Language Support (i18n)**: Supporting regional languages to cater to a broader demographic of tenants and local property managers.

---

*End of Document*